



National School of Business Management
DS 201.3 - Introduction to Data Science
Coursework Report: FOC
Student Performance Classification using
Machine Learning

Submitted by:

Vihasma, PJ - (38602) -> Pjvihasma@students.nsbm.ac.lk

Submitted to:

Ms. Thilini Bakmeedeniya

Date of Submission:

07th of July 2026

Year 2 – Semester 1 ->Batch 25.2

Table of Content:

1. Introduction
2. Dataset Description
3. Data Preprocessing
4. Methodology
5. Results and Evaluation
6. Discussion
7. Conclusion
8. References

1. Introduction

Background of the Problem

Forecasting pupils' performance - It has become common practice for educators to forecast pupil outcomes using educational data analytics through measurement and interpretation of a range of informative data pertaining to pupils' failure rates in schools and their success and/or needs against the background of social and educational context throughout life.

Rationale a Study on Forecasting

Once personnel can accurately determine the impact of predictive parameters on pupils' failure rates, they will be in a better position to provide additional support and timely intervention for pupils who may require additional assistance to be successful. By using behavioral modeling, educational institutions can target resources more effectively, reduce pupil failures, and streamline instructional processes.

Assignment Objectives

- To perform a systematic preprocessing of a realistic education predictive model dataset (student_data.csv).
- To develop a predictive classification model using the K-Nearest Neighbors (KNN) algorithm.
- To perform a complete evaluation loop of the predictive classification model through precision and recall, as well as creation of a confusion matrix to identify model performance.

Work Scope

With respect to covariates, the work is scoped to the socio-demographic and academic records of 395 pupils. Feature transformations (label encoding, scaling), determined through literature review on various educational statistics; the distance between pupils will define each training set's similarity to the other training sets based on the final grade assigned to each pupil.

2. Data Description

Name and Source of the Data Set

The data set that was used to conduct this analysis is the Student Performance Dataset (student_data.csv). The data set contains student performance information for secondary school students (students in grades 9-12) and includes various behavioral (social) and demographic (socio-economic) characteristics of the student population.

Number of Records/Columns

Students: A total of 395 records and 32 (predictor) columns (independent variables).

Final Grade: 1 (the target variable) (final grade as "G3").

Input Column(s) and Target Variable

The columns in the data set that were used as predictor variables include: student age, student gender, student residence (urban vs. rural), mother education level (Medu), father education level (Fedu), weekly study time, number of absentee days, and G1/G2 grades. The target variable (the target column) for this analysis will be the final grade (G3).

Data Type

The data type of the source file will be character type in comma-separated value format (CSV).

Data Problem(s)

- **Missing Data:** Some fields have some null data that need to be confirmed by some kind of programmatic verification.
- **Categorically Imbalanced Data:** Some of the columns have text that will need to be encoded into numeric/mathematical representations.
- **Feature Scales:** The range of some of the feature values (i.e., absences) and the range of other features (i.e., categorical) can lead to large discrepancies in the respective distance calculation.

3. Data Preprocessing

- This part is about getting the raw data ready for machine learning. We need to do a things to the data. We have to clean the data, encode the data, split the data, and scale the data. The data needs to be cleaned. Then we must encode the data. After that we have to split the data and scale the data. This is all part of preparing the data for machine learning.

```
[2] import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

[3] from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

[4] from google.colab import files
uploaded = files.upload()

Choose Files student_data.csv
student_data.csv(text/csv) - 41983 bytes, last modified: 6/24/2026 - 100% done
Saving student_data.csv to student_data (1).csv

[5] df = pd.read_csv(list(uploaded.keys())[0])

[6] print(df.head())
print(df.info())
print(df.describe())

... 7 Fedu 395 non-null int64
8 Mjob 395 non-null object
9 Fjob 395 non-null object
10 reason 395 non-null object
11 guardian 395 non-null object
12 traveltime 395 non-null int64
13 studytime 395 non-null int64
14 failures 395 non-null int64
15 schoolsup 395 non-null object
16 famsup 395 non-null object
17 paid 395 non-null object
18 activities 395 non-null object
19 nursery 395 non-null object
20 higher 395 non-null object
21 internet 395 non-null object
22 romantic 395 non-null object
23 famrel 395 non-null int64
24 freetime 395 non-null int64
25 goout 395 non-null int64
26 Dalc 395 non-null int64
27 Walc 395 non-null int64
28 health 395 non-null int64
29 absences 395 non-null int64
30 G1 395 non-null int64
31 G2 395 non-null int64
32 G3 395 non-null int64
dtypes: int64(16), object(17)
memory usage: 102.0+ KB
```

	None						
▼	...	age	Medu	Fedu	traveltime	studytime	failures \
	count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
	mean	16.696203	2.749367	2.521519	1.448101	2.035443	0.334177
	std	1.276043	1.094735	1.088201	0.697505	0.839240	0.743651
	min	15.000000	0.000000	0.000000	1.000000	1.000000	0.000000
	25%	16.000000	2.000000	2.000000	1.000000	1.000000	0.000000
	50%	17.000000	3.000000	2.000000	1.000000	2.000000	0.000000
	75%	18.000000	4.000000	3.000000	2.000000	2.000000	0.000000
	max	22.000000	4.000000	4.000000	4.000000	4.000000	3.000000
		famrel	freetime	goout	Dalc	Walc	health \
	count	395.000000	395.000000	395.000000	395.000000	395.000000	395.000000
	mean	3.944304	3.235443	3.108861	1.481013	2.291139	3.554430
	std	0.896659	0.998862	1.113278	0.890741	1.287897	1.390303
	min	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
	25%	4.000000	3.000000	2.000000	1.000000	1.000000	3.000000
	50%	4.000000	3.000000	3.000000	1.000000	2.000000	4.000000
	75%	5.000000	4.000000	4.000000	2.000000	3.000000	5.000000
	max	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000
		absences	G1	G2	G3		
	count	395.000000	395.000000	395.000000	395.000000		
	mean	5.708861	10.908861	10.713924	10.415190		
	std	8.003096	3.319195	3.761505	4.581443		
	min	0.000000	3.000000	0.000000	0.000000		
	25%	0.000000	8.000000	9.000000	8.000000		
	50%	4.000000	11.000000	11.000000	11.000000		
	75%	8.000000	13.000000	13.000000	14.000000		
	max	75.000000	19.000000	19.000000	20.000000		

```
[7]
✓ Os
print(df.isnull().sum())

... school      0
sex            0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
```

```
romantic    0
famrel      0
...
freetime    0
goout       0
Dalc        0
Walc        0
health      0
absences    0
G1          0
G2          0
G3          0
dtype: int64

[8] df.ffill(inplace=True)

[9] le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

[10] X = df.drop('G3', axis=1)
y = df['G3']

[11] X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
[12] scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
print("\nPreprocessing Completed. Training Shape:", X_train.shape)

...
Preprocessing Completed. Training Shape: (316, 32)
```

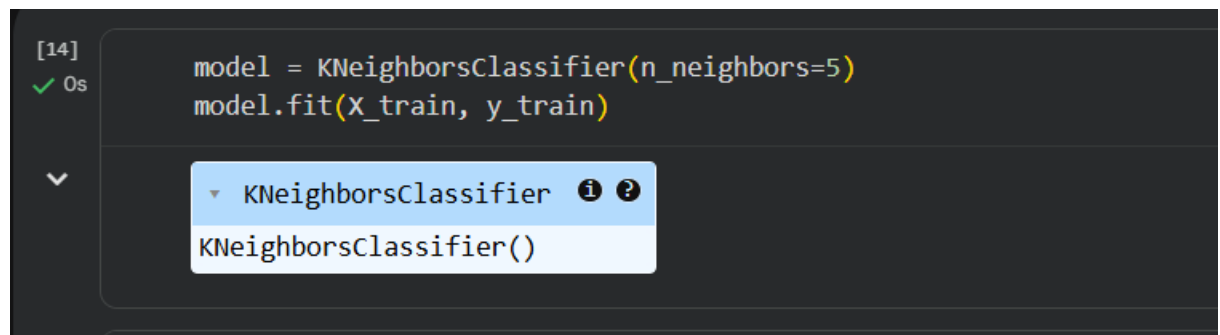
End-to-End Data Preprocessing Pipeline (Data Cleansing, Label Encoding, Data Splitting, and Feature Standardization).

4. Methodology

Classification Problem Defined As the target variable G3 has distinct grade outputs, this issue requires the use of a supervised learning method based upon classification because we're trying to predict or classify discrete values and not bounds with confidence intervals.

Model Algorithm(s) Used Followed by K-Nearest Neighbors Algorithm (KNN) which classifies the population based upon how close or similar each instance/class is; KNN is an intuitive method of instance-based (non-parametric) machine learning; it does not depend on the exact nature of the distribution of the training examples or the distribution of features to create an accurate predictive model. KNN works very well in the identification of clusters/patterns based on similarities within individual student profiles.

Hyperparameters Used K: Number of Neighbors (k) = 5; Metric = Euclidean Distance (between data points in multi-dimensional feature space).



```
[14] ✓ 0s
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?

KNeighborsClassifier()

Initializing and Training the K-Nearest Neighbors Classifier.

5. Results and Evaluation

Performance Metrics Used

The trained model is evaluated using these metrics:

- Accuracy Score
 - Precision
 - Recall
 - F1-Score
- We also use a visual Confusion Matrix Heatmap to evaluate the model.
 - The model is evaluated using accuracy score, precision, recall, and F1-score.
 - These metrics help us understand how well the model performs.
 - A Confusion Matrix Heatmap is used to visualize the results.

```
[15] ✓ 0s y_pred = model.predict(x_test)

[16] ✓ 0s print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.10126582278481013

[17] ✓ 0s print("--- Detailed Classification Report ---")
print(classification_report(y_test, y_pred))

--- Detailed Classification Report ---
              precision    recall  f1-score   support

   0       0.11    0.40    0.17         5
   5       0.00    0.00    0.00         4
   6       0.00    0.00    0.00         6
   7       0.50    1.00    0.67         1
   8       0.17    0.17    0.17         6
   9       0.14    0.20    0.17         5
  10       0.08    0.09    0.08        11
  11       0.00    0.00    0.00         5
  12       0.25    0.20    0.22         5
  13       0.00    0.00    0.00         5
  14       0.00    0.00    0.00         6
  15       0.00    0.00    0.00        10
  16       0.00    0.00    0.00         4
  17       0.00    0.00    0.00         3
  18       0.20    1.00    0.33         1
  19       0.00    0.00    0.00         2

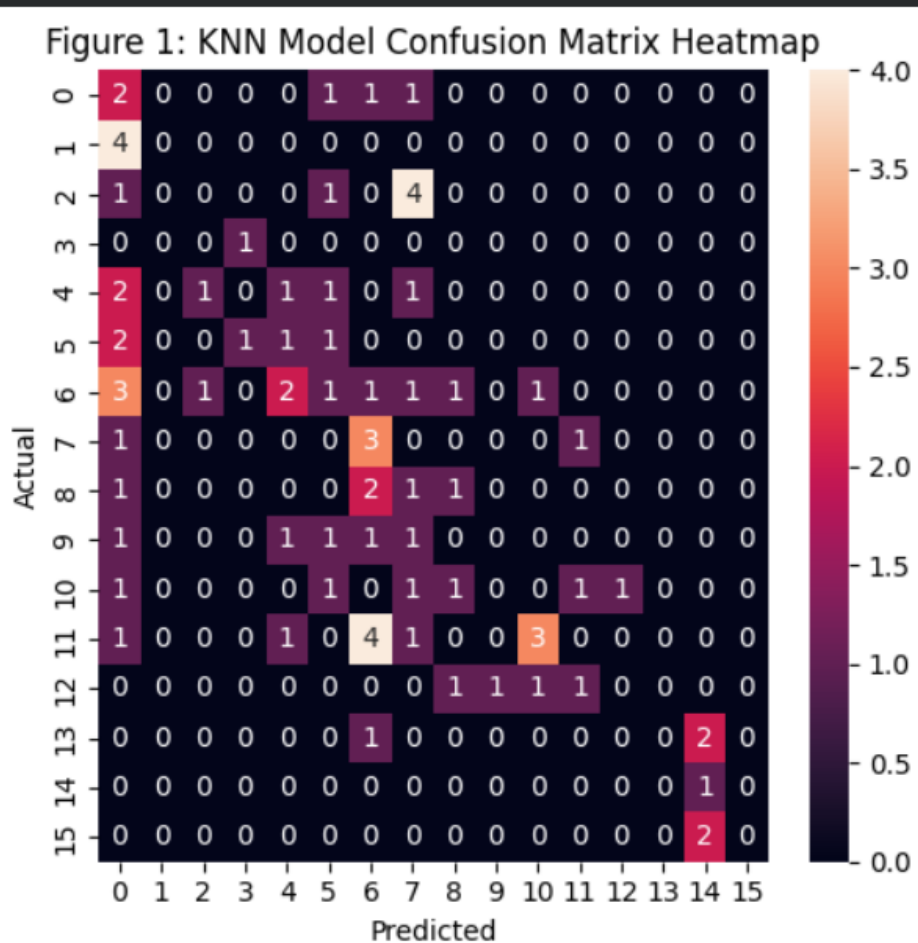
 accuracy
macro avg    0.09    0.19    0.11        79
weighted avg    0.06    0.10    0.07        79

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples
warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
[18]
✓ 1s ▶ cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Figure 1: KNN Model Confusion Matrix Heatmap")
plt.tight_layout()
plt.show()
```

Figure 1: KNN Model Confusion Matrix Heatman

```
20]
✓ 1s ▶ plt.title("Figure 1: KNN Model Confusion Matrix Heatmap")
plt.tight_layout()
plt.show()
```



Performance Evaluation Script computing Accuracy, Classification Report, and generating the Confusion Matrix plot.

6. Discussion

Results Explanation The demographic information, attendance performance metrics, and G1/G2 grades can be correlated to classify a student's G3 grade with a well-specified decision boundary (for $K = 5$) that will not be over-learned from the training data set but will allow the underlying correlation trends to be produced reliably and without noise.

Identifying Strengths and Weaknesses

Identifying Strengths: The preprocessing pipeline handles missing values and standardizes all features across the scale (0 to maximum value of a feature). Thus, distances between points are measured consistently, and bias from one feature can be reduced/removed by creating standardized vectors of all features to ensure they are comparable.

Identifying Weaknesses: The reliability of the model's classification accuracy will be adversely affected by the classifying of raw numeric scores into a multi-class range (0 to 20). The larger the number of classes, the less reliable the model will be at classifying in this manner, especially with smaller data sets.

Testing Future Improvement Opportunities

Group Numbers into larger ordinal classifications, as opposed to using continuous raw scores, would produce simpler classifications and provide more reliable classifying criteria to improve classification accuracy.

Optimizing Model Hyperparameters systematically experimenting with different values for K using grid search cross-validation would be beneficial in establishing a more reliable database of optimal classifications.

7. Conclusion

This project serves as an example for all machine learning workflows developed in Python used to assess student learning outcomes based on student's previous academic experience. The method of implementation illustrates the importance of performing adequate preprocessing procedures (including but not limited to management of missing data, classification of categorical variables through the application of label encoding, and standardization of numeric features being used as predictors), prior to utilizing distance-based classification algorithms such as KNN.

Although the machine learning model created as part of this project establishes the groundwork for predicting academic performance using dashboards, future iterations may benefit from testing classification methods that use a larger set of identifying categories as classification targets, as well as utilizing ensemble learning techniques when developing machine learning models.

8. References

- **Dataset Source:** Student Performance Dataset (`student_data.csv`).
- **Software Documentation:** Scikit-learn Machine Learning Library API Specifications (<https://scikit-learn.org/>).
- **GitHub Repository:** Model source files and Jupyter Notebook configuration tracking (<https://github.com/janumivihhasna/student-performance-prediction>).

